

Portfolio LOTR

El Señor de los Anillos - Documentación Completa

Autor: Luis F. (luisfetrabajo)

Versión: 1.0.0

Licencia: MIT

Generado: Julio 2026

Índice

1. Resumen del Proyecto
2. Stack Tecnológico
3. Estructura de Archivos
4. Configuración de Supabase
5. Base de Datos - Tablas
6. Row Level Security (RLS)
7. Sistema de Autenticación
8. Roles de Usuario
9. Páginas y Rutas
10. Componentes
11. Servicios (Capa de Datos)
12. Hooks Personalizados
13. Estilos y Tema LOTR
14. CI/CD - Deploy
15. Cómo Usar la Autenticación
16. Guía de Administración

1. Resumen del Proyecto

Portfolio LOTR es una aplicación web profesional de tipo portfolio personal, construida con temática de El Señor de los Anillos. Permite al desarrollador\Uy: sus proyectos, habilidades, experiencia y recibir mensajes de contacto.

La aplicación cuenta con un panel de administración completo (CRUD) para gestionar todo el contenido, protegido por autenticación con roles.

Características principales:

- Diseño temático LOTR con animaciones y transiciones
- Panel de administración con CRUD completo
- Autenticación por email/contraseña con roles (admin, authorized, guest)
- Integración con API pública de GitHub para repositorios
- Formulario de contacto con almacenamiento en base de datos
- Tema oscuro/claro con persistencia en localStorage
- Diseño responsive para móviles y escritorio
- Soporte para Android via Capacitor

2. Stack Tecnológico

Frontend:

- React 19.0.0 - Framework de UI
- Ionic React ^8.5.0 - Componentes UI + integración móvil
- React Router DOM ^5.3.4 - Enrutamiento (v5 con Switch)
- TypeScript ~5.9.0 - Tipado estático
- Framer Motion ^12.43.0 - Animaciones declarativas
- Ionicons ^7.4.0 - Iconos Ionic
- Lucide React ^1.27.0 - Iconos modernos

Backend / BaaS:

- Supabase ^2.110.2 - Backend-as-a-Service
- Supabase Auth - Autenticación
- Supabase Storage - Almacenamiento de imágenes
- PostgreSQL - Base de datos relacional

Build / Tooling:

- Vite ^5.0.0 - Bundler y dev server
- @vitejs/plugin-react - Plugin React
- @vitejs/plugin-legacy - Compatibilidad navegadores legacy
- Capacitor ^8.4.2 - Empaquetado nativo Android

Fuentes:

- Cinzel / Cinzel Decorative - Títulos
- Inter - Cuerpo de texto

3. Estructura de Archivos

```
portfolio-lotr/
% % % .env          # Variables de entorno (Supabase + GitHub)
% % % capacitor.config.ts  # Config Capacitor (Android)
% % % index.html     # Entry point HTML
% % % ionic.config.json  # Config Ionic
% % % package.json    # Dependencias
% % % vite.config.ts   # Config Vite
% % % tsconfig.json    # Config TypeScript
% % % supabase/
% % % % schema.sql     # Schema completo BD
% % % % src/
% % % % % main.tsx      # Entry React
% % % % % App.tsx       # Router principal
% % % % % config/
% % % % % % supabase.ts  # Cliente Supabase
% % % % % % context/
% % % % % % % AuthContext.tsx # Autenticación
% % % % % % % ThemeContext.tsx # Tema oscuro/claro
% % % % % % % hooks/      # Hooks custom
% % % % % % % layouts/
% % % % % % % % MainLayout.tsx # Layout principal
% % % % % % % % pages/
% % % % % % % % % HomePage.tsx
% % % % % % % % % SkillsPage.tsx
% % % % % % % % % ProjectsPage.tsx
% % % % % % % % % AboutPage.tsx
% % % % % % % % % ContactPage.tsx
% % % % % % % % % LoginPage.tsx
% % % % % % % % % admin/AdminPage.tsx
% % % % % % % % % components/ # Componentes UI
% % % % % % % % % services/   # Capa de acceso a datos
% % % % % % % % % styles/     # CSS (lotr-theme, animations)
% % % % % % % % % types/      # Interfaces TypeScript
% % % % % % % % % utils/      # Constantes
```

4. Configuración de Supabase

El cliente Supabase se inicializa en `src/config/supabase.ts` usando variables de entorno:

Variables de entorno (.env):

- `VITE_SUPABASE_URL`: URL del proyecto Supabase
 - `VITE_SUPABASE_PUBLISHABLE_KEY`: Clave pública (anon key)
 - `VITE_GITHUB_USERNAME`: Usuario de GitHub para la API
-

Conexión:

```
const url = import.meta.env.VITE_SUPABASE_URL;  
const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;  
export const supabase = createClient(url, key);
```

5. Base de Datos - Tablas

5.1 usuarios

Almacena perfiles de usuario vinculados a auth.users de Supabase.

- id: UUID (PK, FK !' auth.users.id)
- email: TEXT NOT NULL
- role: TEXT (admin | authorized | guest)
- full_name: TEXT
- created_at: TIMESTAMPTZ

Trigger automático: Al crear usuario en auth.users, se inserta en usuarios con rol "guest".

5.2 proyectos

Almacena los proyectos del portfolio.

- id: UUID (PK)
- titulo: TEXT NOT NULL
- descripcion: TEXT
- imagen: TEXT (URL de imagen)
- github: TEXT (URL del repositorio)
- demo: TEXT (URL de demo)
- tecnologias: TEXT[] (array de tecnologías)
- estado: TEXT (en_desarrollo | finalizado | archivado)
- fecha: DATE
- destacado: BOOLEAN
- created_at: TIMESTAMPTZ

5.3 habilidades

- id: UUID (PK)
- nombre: TEXT NOT NULL
- icono: TEXT (emoji por defecto Ø=Ý')
- categoria: TEXT (frontend | backend | herramientas)

5.4 experiencia

- id: UUID (PK)
- empresa: TEXT NOT NULL
- cargo: TEXT NOT NULL
- descripcion: TEXT
- fecha_inicio: DATE NOT NULL
- fecha_fin: DATE (NULL = presente)

5.5 certificados

- id: UUID (PK)
- titulo: TEXT NOT NULL
- institucion: TEXT NOT NULL
- url: TEXT
- imagen: TEXT

5.6 mensajes

Mensajes enviados desde el formulario de contacto.

- id: UUID (PK)
- nombre: TEXT NOT NULL
- email: TEXT NOT NULL
- asunto: TEXT NOT NULL
- mensaje: TEXT NOT NULL
- leído: BOOLEAN (default false)
- created_at: TIMESTAMPTZ

6. Row Level Security (RLS)

Todas las tablas tienen RLS habilitado. Políticas por tabla:

proyectos / habilidades:

- SELECT: Público (cualquiera puede leer)
- INSERT/UPDATE/DELETE: Solo admin

experiencia / certificados:

- SELECT: Admin y authorized
- INSERT/UPDATE/DELETE: Solo admin

mensajes:

- INSERT: Público (cualquiera puede enviar)
- SELECT/UPDATE/DELETE: Solo admin

usuarios:

- SELECT: Cada usuario lee su propio perfil + admin lee todos
- UPDATE: Solo admin

Storage (bucket "projects"):

- SELECT: Público
- INSERT/DELETE: Solo admin

7. Sistema de Autenticación

La autenticación está implementada con Supabase Auth usando email y contraseña.

Flujo de autenticación:

1. El usuario ingresa email y contraseña en /login
 2. Se llama a `supabase.auth.signInWithPassword()`
 3. Supabase valida las credenciales y retorna una sesión
 4. El `AuthContext` guarda el usuario y busca su perfil en "usuarios"
 5. Se determina el rol (admin, authorized, guest)
 6. Si es admin, se redirige a /admin
-

Archivos clave:

- `src/config/supabase.ts` - Cliente Supabase
 - `src/context/AuthContext.tsx` - Provider de autenticación
 - `src/pages/LoginPage.tsx` - Formulario de login
-

AuthContext expone:

- `user`: Objeto User de Supabase
- `profile`: Perfil completo de la tabla usuarios
- `role`: Rol del usuario (admin | authorized | guest)
- `loading`: Estado de carga
- `signIn(email, password)`: Función para iniciar sesión
- `signOut()`: Función para cerrar sesión
- `isAdmin`: Boolean - es admin?
- `isAuthorized`: Boolean - es admin o authorized?

8. Roles de Usuario

admin

- Acceso total al panel de administración
- CRUD completo: crear, editar, eliminar proyectos, habilidades, mensajes
- Puede subir/eliminar imágenes en Storage
- Puede ver y gestionar todos los mensajes
- Puede ver experiencia y certificados

authorized

- Puede ver experiencia y certificados
- NO puede acceder al panel de administración
- NO puede modificar contenido

guest

- Solo puede ver contenido público
- Proyectos, habilidades, página principal
- NO ve experiencia ni certificados
- Rol por defecto al registrarse

Cambio de rol:

Los roles se gestionan directamente en Supabase Dashboard > Table Editor > usuarios. Solo un admin puede cambiar el rol de otro usuario ejecutando SQL:

```
UPDATE usuarios SET role = 'admin' WHERE email = 'usuario@email.com';
```

9. Páginas y Rutas

- /!' HomePage
Hero animado, tech stack, proyectos destacados, repos de GitHub
- /habilidades !' SkillsPage
Grid de habilidades con filtro por categoría
- /proyectos !' ProjectsPage
Proyectos con búsqueda, filtros y ordenamiento
- /sobre-mi !' AboutPage
Perfil de GitHub, stats del desarrollador
- /contacto !' ContactPage
Formulario de contacto (envía a Supabase)
- /login !' LoginPage
Login de administrador (email + contraseña)
- /admin !' AdminPage
Panel de control con 4 tabs (solo admin)

AdminPage - Tabs:

- Dashboard: Estadísticas generales
- Proyectos: CRUD de proyectos
- Habilidades: CRUD de habilidades
- Mensajes: Ver, marcar leído, eliminar mensajes

10. Componentes

- HeroSection: Hero animado con icono de anillo, badges de tecnología, botones CTA
- Header: Barra de navegación con Ionic (IonHeader/IonToolbar)
- Footer: Pie de página con enlaces y easter egg (5 clics = confetti)
- ProjectCard: Tarjeta de repositorio GitHub con info detallada
- TechBadge: Badge animado con icono y nombre de tecnología
- TechTree: Grid de tecnologías con nivel y lore LOTR
- GithubRepos: Lista de repos con filtro por lenguaje

11. Servicios (Capa de Datos)

- ProjectService !' Tabla: proyectos
Métodos: getAll, getFeatured, getById, create, update, delete, toggleFeatured
- SkillService !' Tabla: habilidades
Métodos: getAll, getByCategory, create, update, delete
- MessageService !' Tabla: mensajes
Métodos: getAll, getUnreadCount, send, markAsRead, delete
- GithubService !' Tabla: API GitHub
Métodos: getRepos (filtrados, sin forks), getUser
- StorageService !' Tabla: Storage
Métodos: uploadImage, deleteImage, getPublicUrl
- ProfileService !' Tabla: profiles
Métodos: getProfile, updateProfile

12. Hooks Personalizados

- `useScrollReveal`: Detecta visibilidad de elemento (`IntersectionObserver`)
- `useDebounce`: Retrasa un valor por tiempo determinado
- `useLocalStorage`: State sincronizado con `localStorage`
- `useClickCounter`: Cuenta clics y ejecuta callback al alcanzar umbral (easter egg)

13. Estilos y Tema LOTR

Paleta de colores:

- Dorado: #c9a227
 - Plata: #c0c0c0
 - Mithril: #b4c5d4
 - Fondo oscuro: #0f0f1a
 - Fondo claro: #f5f5f5
-

Archivos CSS:

- lotr-theme.css (~980 líneas) - Estilos principales
 - animations.css - Keyframes y clases utilitarias
 - variables.css - Variables CSS de Ionic + tema
-

Características visuales:

- Tema oscuro por defecto con toggle a claro
- Persistencia del tema en localStorage
- Animaciones: flotación del anillo, fade-up, shimmer dorado
- Skeleton loaders para carga de datos
- Tarjetas con gradientes y bordes dorados
- Nombres temáticos LOTR en secciones

14. CI/CD - Deploy

GitHub Actions automatiza el build y deploy:

Trigger:

- Push a rama main
- Ejecución manual (workflow_dispatch)

Pipeline:

- 1. Checkout del código
 - 2. Setup Node.js 20
 - 3. npm ci (instalación limpia)
 - 4. npm run build con secrets de Supabase
 - 5. Deploy a GitHub Pages
-

Secrets requeridos:

- VITE_SUPABASE_URL
- VITE_SUPABASE_PUBLISHABLE_KEY
- VITE_GITHUB_USERNAME

15. Cómo Usar la Autenticación

Paso 1: Crear usuario en Supabase

Ve a Supabase Dashboard > Authentication > Users > Add User

- Email: tu@email.com
 - Password: tu contraseña
 - Marca "Auto Confirm Email"
-

Paso 2: Asignar rol de admin

Ve a Supabase Dashboard > SQL Editor y ejecuta:

```
UPDATE usuarios SET role = 'admin' WHERE email = 'tu@email.com';
```

Esto cambia el rol del usuario de "guest" a "admin".

Paso 3: Iniciar sesión

1. Navega a /login
 2. Ingresa email y contraseña
 3. Si las credenciales son correctas, serás redirigido a /admin
 4. El navbar mostrará el link "Admin" solo si eres admin
-

Paso 4: Gestionar contenido

Desde /admin puedes:

- Crear, editar y eliminar proyectos
- Crear, editar y eliminar habilidades
- Ver y gestionar mensajes de contacto
- Ver estadísticas del dashboard

16. Guía de Administración

Crear proyecto:

1. Ve a /admin > tab Proyectos
 2. Haz clic en "Nuevo Proyecto"
 3. Completa: título, descripción, GitHub URL, demo URL
 4. Selecciona tecnologías y estado
 5. Marca como destacado si quieres que aparezca en el TM—~u
-

Crear habilidad:

1. Ve a /admin > tab Habilidades
 2. Haz clic en "Nueva Habilidad"
 3. Ingresa nombre, icono (emoji) y categoría
-

Gestionar mensajes:

1. Ve a /admin > tab Mensajes
 2. Los mensajes nuevos aparecen sin leer
 3. Haz clic en un mensaje para verlo completo
 4. Marca como leído o elimínalo
-

Subir imágenes:

Las imágenes de proyectos se suben a Supabase Storage bucket "projects".
Solo los admin pueden subir/eliminar archivos.

¡Proyecto Listo!

La autenticación ya está implementada y funcional.
Solo necesitas crear tu usuario en Supabase y asignar el rol de admin.

Luis F. - [luisfetrabajo](https://luisf.dev)
<https://luisf.dev>